

Chapter 15 - Software Design Patterns

Software Design Patterns with examples

STRUCTURAL

1) Adapter

Convert the interface of a class into another interface clients expect. It lets the classes work together that couldn't otherwise because of incompatible interfaces.

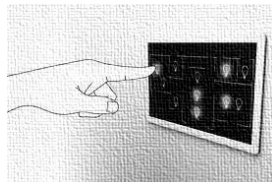
Socket attaches to a Ratchet



2) Bridge

Decouple an abstraction from its implementation so that the two can vary independently.

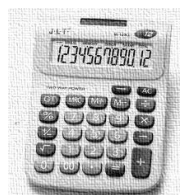
Switch controlling lights



3) Composite

Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.

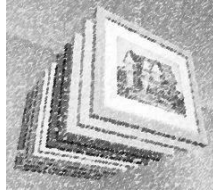
Arithmetic Expression



4) Decorator

Attach additional responsibilities to objects dynamically. Provide a flexible alternative to sub-classing for extending functionality.

Paintings with Frames



5) Facade

Provide a unified interface to a set of interfaces in a subsystem. It defines a high-level interface that makes the subsystem easier to use.

Customer Service Representative



6) Flyweight

Use sharing to support large number of fine grained objects effectively.

Public Switched Telephone Network



7) Proxy

Provide a surrogate or placeholder for another object to control access to it.

Check or Bank Draft



CREATIONAL

8) Abstract Factory

It provides an interface for creating families of related or dependent objects without specifying their concrete class.

Stamping Equipment



9) Builder

Separate the construction of complex object from its representing so that the same construction process can create different representations.

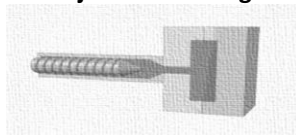
Kids Meal



10) Factory Method

Define an interface for creating an object, but let subclasses decide which class to instantiate. Let the class defer Instantiation to subclasses.

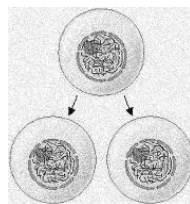
Injection Molding



11) Prototype

Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

Division of a cell – Cloning



12) Singleton

Ensure a class only has one instance and provide a global point of access to it.

Office of the president of the United States



BEHAVIORAL

13) Chain of Responsibility

Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.

Coin Sorting Machine



14) Command

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

Diner Check



15) Interpreter

Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.

Musicians



16) Iterator

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Modern Television Sets



17) Mediator

Define an object that encapsulates how a set of objects interact. It promotes loose coupling by keeping objects from referring to each other explicitly and lets you to vary their interactions independently.

Airport Control Tower



18) Memento

Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.

Furniture Assembling Manual



19) Observer

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Auction



20) State

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

ATM or Vending Machine



21) Strategy

Define a family of algorithms, encapsulate each one, and make them interchangeable. Let the algorithm vary independently from clients that use it.

Modes of Transportation to a place



22) Template

Define the Skelton of an algorithm in an operation, deferring some steps to subclasses. Let subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

House Floor Plan

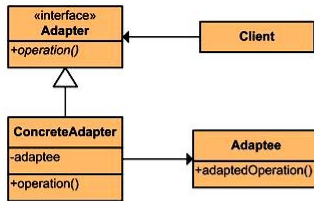


23) Visitor

Represent an operation to be performed on the elements of an object structure. Lets you define a new operation without changing the classes of the elements on which it operates.

Operation of a Taxi Company

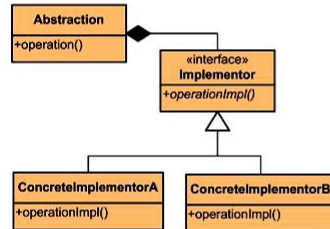




Adapter

Type: Structural

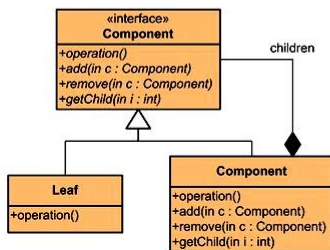
What it is: Convert the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.



Bridge

Type: Structural

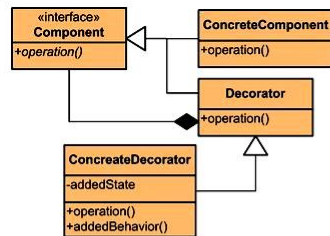
What it is: Decouple an abstraction from its implementation so that the two can vary independently.



Composite

Type: Structural

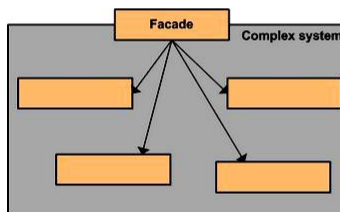
What it is: Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.



Decorator

Type: Structural

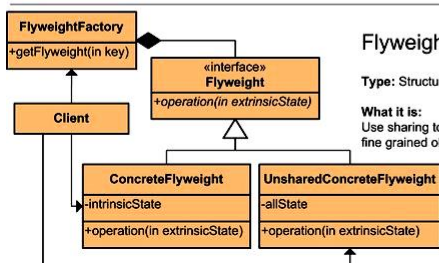
What it is: Attach additional responsibilities to an object dynamically. Provide a flexible alternative to sub-classing for extending functionality.



Facade

Type: Structural

What it is: Provide a unified interface to a set of interfaces in a subsystem. Defines a high-level interface that makes the subsystem easier to use.



Flyweight

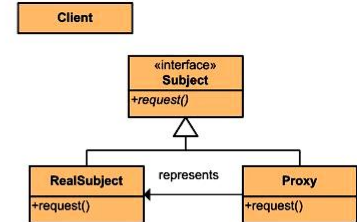
Type: Structural

What it is: Use sharing to support large numbers of fine grained objects efficiently.

Proxy

Type: Structural

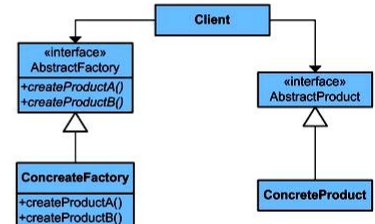
What it is: Provide a surrogate or placeholder for another object to control access to it.



Abstract Factory

Type: Creational

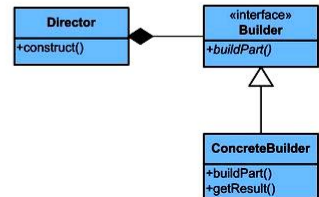
What it is: Provides an interface for creating families of related or dependent objects without specifying their concrete class.



Builder

Type: Creational

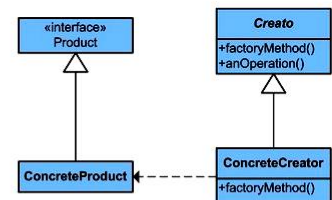
What it is: Separate the construction of a complex object from its representing so that the same construction process can create different representations.



Factory Method

Type: Creational

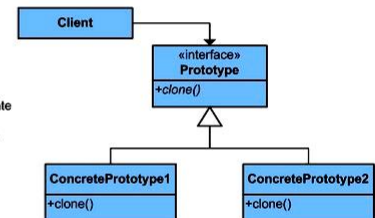
What it is: Define an interface for creating an object, but let subclasses decide which class to instantiate. Lets a class defer instantiation to subclasses.



Prototype

Type: Creational

What it is: Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.



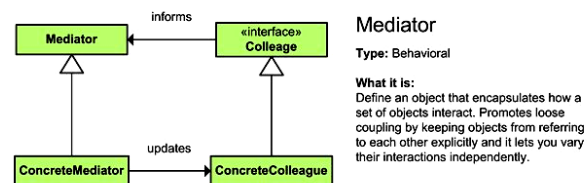
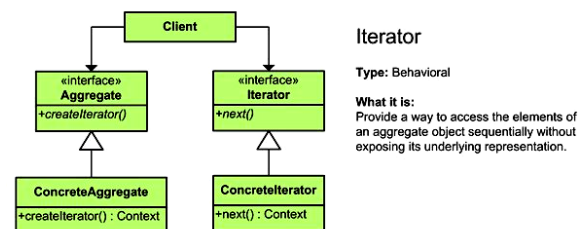
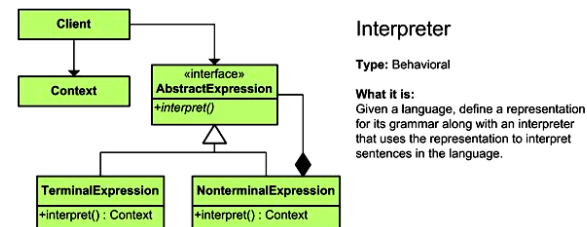
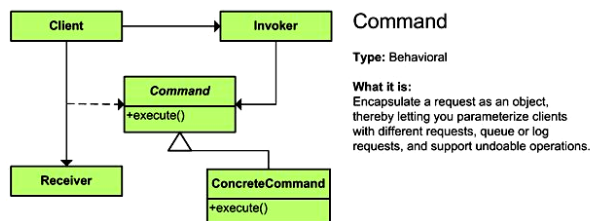
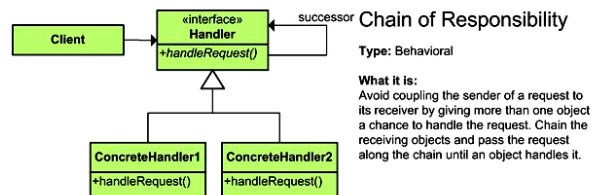
Singleton

Type: Creational

What it is: Ensure a class only has one instance and provide a global point of access to it.



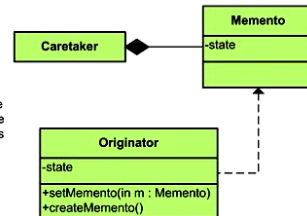
C	Abstract Factory	S	Facade	S	Proxy
S	Adapter	C	Factory Method	B	Observer
S	Bridge	S	Flyweight	C	Singleton
C	Builder	B	Interpreter	B	State
B	Chain of Responsibility	B	Iterator	B	Strategy
B	Command	B	Mediator	B	Template Method
S	Composite	B	Memento	B	Visitor
S	Decorator	C	Prototype		



Memento

Type: Behavioral

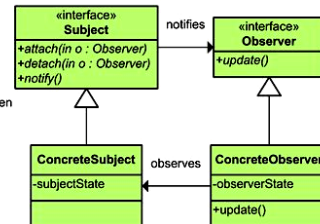
What it is:
Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.



Observer

Type: Behavioral

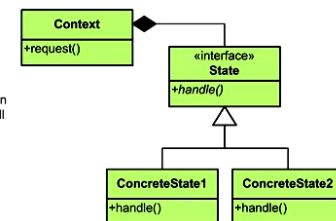
What it is:
Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



State

Type: Behavioral

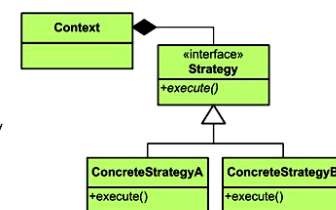
What it is:
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.



Strategy

Type: Behavioral

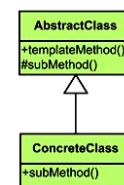
What it is:
Define a family of algorithms, encapsulate each one, and make them interchangeable. Lets the algorithm vary independently from clients that use it.



Template Method

Type: Behavioral

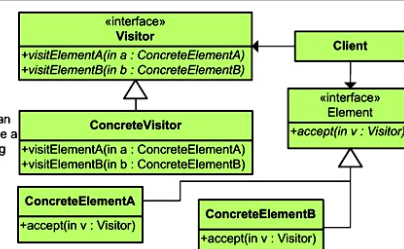
What it is:
Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.



Visitor

Type: Behavioral

What it is:
Represent an operation to be performed on the elements of an object structure. Lets you define a new operation without changing the classes of the elements on which it operates.



Chapter 16 – J2EE Design Patterns with Frameworks

J2EE Design Patterns with Frameworks

Well, the design pattern is simply a description of a recurring solution to a problem, given a context. The context is the environment, surroundings, situation, or interrelated conditions within which the problem exists.

Design patterns have a number of advantages like

1. Once described, any level engineer can use the pattern.
2. They allow for reuse without having to reinvent in every a project.
3. They allow to better defining system structure.
4. They provide a design vocabulary.
5. They provide reusable artifacts.
6. Patterns can form frameworks that can then be used for implementations.

In this section we have tried to cover how to use and identify design patterns, in J2EE applications. The interest in design patterns has been around for a number of years in the software industry. However, interest among mainstream software developers is a fairly recent development actually it takes a highly experienced engineer to recognize a pattern, it requires collaboration, and it requires ongoing refinements.

There are a number of patterns that have been identified by the Sun Java Center for the presentation tier.

1. Intercepting Filter

It facilitates preprocessing and post-processing of a request.

2. Front Controller

It provides a centralized controller for managing the handling of requests.

3. Composite View

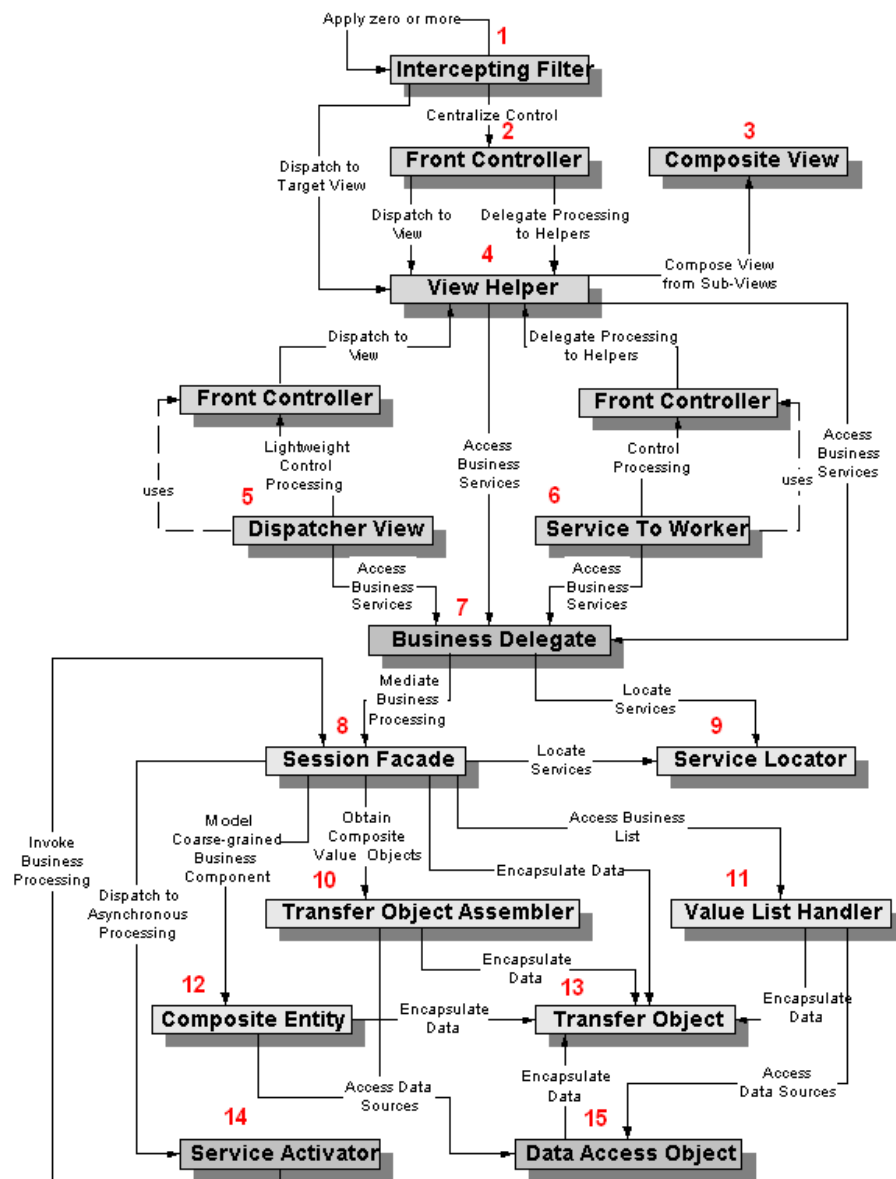
It creates an aggregate View from atomic subcomponents.

4. View Helper

It encapsulates logic that is not related to presentation formatting into Helper components.

5. Dispatcher View

It combines a Dispatcher component with the Front Controller and View Helper patterns, deferring many activities to View processing.



6. Service to Worker

It combines a Dispatcher component with the Front Controller and View Helper patterns. Service to Worker pattern usually works with Front Controller. The Front Controller is responsible to delegate the request to a worker module.

Dispatcher View pattern works the same way but the **opposite direction** of Service to Worker pattern.

7. Business Delegate

It reduces coupling between presentation-tier clients and business services. It hides the underlying implementation details of the business service, such as lookup and access details of the EJB architecture.

8. Session Facade

It encapsulates the complexity of interactions between the business objects participating in a workflow. The Session Facade manages the business objects, and provides a uniform coarse-grained service access layer to clients.

9. Service Locator

Multiple clients can reuse the Service Locator object to reduce code complexity, provide a single point of control, and improve performance by providing a caching facility.

10. Transfer Object Assembler

It is used to build the required model or sub model. The Transfer Object Assembler uses Transfer Objects to retrieve data from various business objects and other objects that define the model or part of the model.

11. Value List Handler

The most critical concern in a distributed paradigm is the latency time. Value List Handler Pattern suggests an alternate approach of using EJB finder methods. The pattern is used to control the search, cache the results and provide the results to the client using a lightweight mechanism.

12. Composite Entity

It model, represent, and manage a set of interrelated persistent objects rather than representing them as individual fine-grained entity beans. A Composite Entity bean represents a graph of objects.

13. Transfer Object

It encapsulates the business data. A single method call is used to send and retrieve the Transfer Object. When the client requests the enterprise bean for the business data, the enterprise bean can construct the Transfer Object, populate it with its attribute values, and pass it by value to the client.

14. Service Activator

It enables asynchronous access to enterprise beans and other business services. It receives asynchronous client requests and messages. On receiving a message, the Service Activator locates and invokes the necessary business methods on the business service components to fulfill the request asynchronously. In EJB2.0, Message Driven beans can be used to implement Service Activator for message based enterprise applications. The Service Activator is a JMS Listener and delegation service that creates a message façade for the EJBs.

15. Data Access Object

Abstracts and encapsulate all access to the data source. The DAO manages the connection with the data source to obtain and store data.

Design Patterns in Spring Framework

Delegation Pattern

In software engineering, the delegation pattern is a design pattern in object-oriented programming where an object, instead of performing one of its stated tasks, delegates that task to an associated helper object. There is an Inversion of Responsibility in which a helper object, known as a delegate, is given the responsibility to execute a task for the delegator. The delegation pattern is one of the fundamental abstraction patterns that underlie other software patterns such as composition (also referred to as aggregation), mixins and aspects.

```
class RealPrinter { // the "delegate"
    void print() {
        System.out.print("something");
    }
}

class Printer { // the "delegator"
    RealPrinter p = new RealPrinter(); // create the delegate
    void print() {
        p.print(); // delegation
    }
}

public class Main {
    // to the outside world it looks like Printer actually prints.
    public static void main(String[] args) {
        Printer printer = new Printer();
        printer.print();
    }
}
```

Delegation is the simple yet powerful concept of handing a task over to another part of the program. In object-oriented programming it is used to describe the situation where one object defers a task to another object, known as the delegate. This mechanism is sometimes referred to as aggregation, consultation or forwarding (when a wrapper object doesn't pass itself to the wrapped object).

Delegation is dependent upon dynamic binding, as it requires that a given method call can invoke different segments of code at runtime.

Spring's Inversion of Control (IOC), also known as Dependency Injection is achieved by applying Delegation pattern. Applying IoC, objects are given their dependencies at creation time by some external entity that coordinates each object in the system. That is, dependencies are injected into objects. So, IoC means an inversion of responsibility with regard to how an object obtains references to collaborating objects.

There are three types of dependency injection:

- **Constructor Injection** (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- **Setter Injection** (e.g. Spring): Dependencies are assigned through JavaBeans properties (ex: setter methods).
- **Interface Injection** (e.g. Avalon): Injection is done through an interface.

Note: Spring supports only Constructor and Setter Injection

The below code snippet explains the constructor and setter dependency injection.

```
class A {  
    void foo() {  
        // "this" also known under the names "current", "me" and "self" in other languages  
        this.bar();  
    }  
  
    void bar() {  
        print("a.bar");  
    }  
};  
  
class B {  
    private A a; // delegation link  
  
    public B(A a) // Constructor Injection    public void setA(A a) // Setter Injection  
    {  
        this.a = a;  
    }  
    {  
        this.a = a;  
    }  
  
    void foo() {  
        a.foo(); // call foo() on the a-instance  
    }  
  
    void bar() {  
        print("b.bar");  
    }  
};  
  
a = new A();  
b = new B(a); // establish delegation between two objects
```

Other Patterns

Spring DAO module uses Data Access Object pattern and Spring MVC module is based on MVC2 architecture.

Design Patterns in Struts Framework

Struts is based on MVC (Model-View-Controller) Model2 or simply **MVC2** architecture. Struts controller uses the command design pattern and the action classes use the adapter design pattern. The process() method of the Request Processor uses the template method design pattern. Struts also implement the following J2EE design patterns.

- **Service to Worker**
- **Dispatcher View**
- **Composite View (Struts Tiles)**
- **Front Controller**
- **View Helper**
- **Synchronizer Token**

An Action Class in Struts performs a role of an **Adapter** between the contents of an incoming HTTP request and the corresponding business logic that should be executed to process this request. Action Servlet of Struts is part of Controller components which works as **Front Controller** to handle all the requests. The Interceptors in Struts 2 uses the **Intercepting Filter** pattern.

Design Patterns in Hibernate Framework

Composite Pattern and Composite Entity Pattern

Hibernate framework implements composite design pattern with annotations to build hierarchical tree structures of entities. It helps creating persistent trees of objects.

Proxy Pattern

The Lazy Loading concept in Hibernate framework is based on proxy collections.

Design Patterns in Apache Axis Framework

Chain of Responsibility Pattern

The chains between the request and response handlers in Apache Axis architecture uses the Chain of Responsibility pattern in which a request flows along a sequence of Handlers until it is processed. Although an Axis Chain may process a request in stages over a succession of Handlers, it has the same advantages as Chain of Responsibility: flexibility and the ease with which new function can be added.